



ЛЕКЦІЯ №4

РЯДКИ

Доступ до елементів,
зрізи, об'єднання,
реєстри, заміна
символів

СТРУКТУРА РЯДКА

Рядок – набір символів UNICODE оточені або одинарними, або подвійними лапками. Для відображення символів використовують функцію PRINT()

```
print("HELLO")  
print('Hello')
```

Багаторядкові рядки

Ви можете призначити багаторядковий рядок змінній, використовуючи три лапки:

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""  
print(a)
```

UNICODE

<http://www.unicode.org/charts/>

ord() – визначає код за символом
chr() – визначає символ за кодом

```
print(chr (8605))  
print(ord("M"))
```

```
PS C:\Users\User> python D:\Python\u1.py  
~  
1052
```

Стрілки

Десятковий код	Шістнадцятеричний код	Символ	Символьне позначення
←	←	←	←
↑	↑	↑	↑
→	→	→	→
↓	↓	↓	↓
↔	↔	↔	↔
↕	↕	↕	
↖	↖	↖	
↗	↗	↗	
↘	↘	↘	
↙	↙	↙	
↚	↚	↖↗	
↛	↛	↗↘	
↜	↜	↻	
↝	↝	↻	

ДОСТУП ДО СИМВОЛІВ РЯДКА

0	1	2	3	4	5	6	7	8	9	10	11	12
H	E	L	L	O	,		W	O	R	L	D	!

```
a = "HELLO, WORLD!"
```

```
print(a[1])
```

```
E
```

Довжина рядка

Щоб отримати довжину рядка, використовуйте **len()** функцію.

```
a = "Hello, World!"
```

```
print(len(a))
```

```
13
```

ДІЇ НАД РЯДКАМИ

Конкатенація рядків За допомогою оператора `+` можна об'єднати рядки або рядкові змінні

```
>>> age = 23
>>> message = "Happy " + str(age) + "rd Birthday!"
>>> print(message)
Happy 23rd Birthday!
```

Дублювання рядків

Оператор `*` (зірочка) можна використовувати для того, щоб продублювати рядок

```
>>> start = 'Na' * 4 + '\n'
>>> middle = 'Hey ' * 3 + '\n'
>>> end = 'Goodbye.'
>>> print(start + start + middle + end)
NaNaNaN
NaNaNaN
Hey Hey Hey
Goodbye.
```

ДОСТУП ДО ЕЛЕМЕНТА РЯДКА ЗА ІНДЕКСОМ

Для того, щоб отримати один символ рядка, задайте індекс (місцезнаходження) цього символу у рядку всередині квадратних дужок після імені рядка:

```
>>> letters = 'abcdefghijklmnopqrstuvwxyz'
>>> letters[0]
'a'
>>> letters[6]
'g'
>>> letters[-1]
'z'
>>> letters[36]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

Індекси мають значення в діапазоні від 0 до довжина рядка - 1.

ЗРІЗИ: ФУНКЦІЯ **`Slice[START: END: STEP]`**

З рядка можна «витягти» не лише один символ, але й підрядок (частину рядка) за допомогою функції `slice`. У квадратних дужках запису цієї функції вказується:

- індекс початку підрядка `start`
- індекс кінця підрядка `end`
- розміру кроку `step`

Деякими з цих параметрів можна знехтувати. У підрядок будуть включені символи, розташовані починаючи з символа, на який вказує індекс `start`, і закінчуватися символом, на який вказує індекс `end`.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z

```

>>> letters = 'abcdefghijklmnopqrstuvwxyz'
>>> letters[:] 1
'abcdefghijklmnopqrstuvwxyz'
>>> letters[20:] 2
'uvwxyz'
>>> letters[12:15] 3
'mno'
>>> letters[-5:] 4
'vwxyz'
>>> letters[18:-3] 5
'stuvw'
>>> letters[-6:-2] 6
'uvwx'
>>> letters[:7] 7
'ahov'
>>> letters[4:20:3] 8
'ehknqt'
>>> letters[19::4] 9
'tx'
>>> letters[:21:5] 10
'afkpu'
>>> letters[::-1] 11
'zyxwvutsrqponmlkjihgfedcba'
>>> letters[60:70] 12
''

```

1. Отримати послідовність символів від початку і до кінця.
2. Отримати послідовність символів від 20-го символу і до кінця.
3. Отримати послідовність символів від 12-го символу по 15-й (15-й символ не включається).
4. Отримати послідовність останніх 5 символів.
5. Отримати послідовність символів від 18-го символу і до 3-го з кінця (3-й символ з кінця не включається).
6. Отримати послідовність символів від 6-го символу з кінця і до 2-го символу з кінця (2-й символ з кінця не включається).
7. Отримати кожний 7-й символ з початку і до кінця.
8. Отримати кожний 3-й символ від 4-го символу з початку і до 20-го символу (20-й символ не включається).
9. Отримати кожний 4-й символ починаючи від 19-го символу і до кінця.
10. Отримати кожний 5-й символ починаючи з початку і до 21-го символу (21-й символ не включається).
11. Перевернути рядок з кінця на початок.

РОЗДІЛЕННЯ РЯДКА: ФУНКЦІЯ **SPLIT()**

Функція **split()** розбиває рядок на окремі рядки і розміщує їх у списку.

Синтаксис

string.split(separator, maxsplit)

separator - визначає роздільник для використання під час розділення рядка. За замовчуванням будь-який пробіл є роздільником

maxsplit - визначає, скільки розділень потрібно зробити. Значення за замовчуванням -1, тобто "всі випадки"

```
txt = "apple#banana#cherry#orange"
```

```
x = txt.split("#", 1)
```

```
['apple', 'banana#cherry#orange']
```

ОБ'ЄДНАННЯ РЯДКІВ: ФУНКЦІЯ **JOIN()**

Функція **join()** об'єднує список рядків в один рядок.

Синтаксис

string.join(iterable)

iterable - Будь-який повторюваний об'єкт, де всі повернуті значення є рядками

```
myTuple = ("John", "Peter", "Vicky")
```

```
x = "#".join(myTuple)
```

```
print(x)
```

John#Peter#Vicky

МЕТОДИ

strip() метод видаляє будь-які початкові (пробіли на початку) і кінцеві (пробіли в кінці) символи (пробіл є початковим символом за замовчуванням, який потрібно видалити)

Синтаксис

string.strip(characters)

```
txt = " ,,,,rrttgg....banana....rrr"  
x = txt.strip(",.grt")  
print(x)
```

banana

МЕТОДИ

rstrip() видаляє будь-які кінцеві символи (символи в кінці рядка), пробіл є кінцевим символом за замовчуванням, який потрібно видалити.

```
txt = "banana,,,,,ssqqqww...."  
  
x = txt.rstrip(",.qsw")
```

lstrip() видаляє будь-які початкові символи (пробіл є початковим символом за замовчуванням для видалення)

```
txt = " ,,,,ssaaww....banana"  
  
x = txt.lstrip(",.asw")
```

МЕТОД **FIND()**

Метод **find()** знаходить перше входження вказаного значення.

Метод **find()** повертає -1, якщо значення не знайдено.

Синтаксис

```
string.find(value, start, end)
```

```
txt = "Hello, welcome to my world."
```

```
x = txt.find("e", 5, 10)
```

РЕГІСТР І ВИРІВНЮВАННЯ

Напишемо всі слова з великої літери - **title()**

Напишемо перше слово з великої літери, а всі інші символи - малими літерами **capitalize()**

Напишемо всі слова великими літерами - **upper()**

Напишемо всі слова малими літерами - **lower()**

Змінимо регістр літер на протилежний - **swapcase()**

КІЛЬКІСТЬ ПІДРЯДКІВ В РЯДКУ

Метод **count()** повертає кількість разів, коли вказане значення з'являється в рядку.

Синтаксис

string.count(value, start, end)

```
txt = "I love apples, apple are my favorite fruit"  
x = txt.count("apple")  
print(x)  
>>>2
```

ЗАМІНА СИМВОЛІВ

Метод **replace()** замінює вказану фразу іншою зазначеною фразою

Синтаксис

string.replace(oldvalue, newvalue, count)

```
txt = "one one was a race horse, two two was one too."  
x = txt.replace("one", "three", 2)  
print(x)
```


ПЕРЕВІРКА НАЯВНОСТІ

Метод **isalnum()** повертає значення `True`, якщо всі символи є алфавітно-цифровими, що означає літеру алфавіту (a z) і цифри (0-9).

Метод **isalpha()** повертає `True`, якщо всі символи є літерами алфавіту (a z)

Метод **isdecimal()** повертає `True`, якщо всі символи є десятковими (0-9).

ПІДХІД ДЛЯ ЗЧИТУВАННЯ ДАНИХ ЗАПИСАНИХ У СТРІЧЦІ

Цей підхід використовується для швидкого введення кількох значень в одному рядку, **розділених пробілами (або іншими символами)**, та їх подальшого перетворення на числа.

Конструкція зчитування `a, b = map(int, input().split())`

Аналіз конструкції

`input()` – зчитує введений користувачем рядок із клавіатури.

`.split()` – метод рядків, який розбиває суцільний рядок на список менших рядків (підрядків) за замовчуванням по пробілах.

`map(int, ...)` – застосовує функцію `int()` до кожного елемента отриманого списку, перетворюючи їх на цілі числа.

`a, b = ...` – розпаковує отримані числа у змінні `a` та `b`.

ЛІТЕРАТУРА

<https://pythonguide.rozh2sch.org.ua>

https://www.w3schools.com/python/python_strings_methods.asp



THE END